PRODUCT REQUIREMENTS DOCUMENT

ClinEdOps — Unified Clinical Education Operations Platform

National University Hospital (NUH) — Medical Affairs (Education)

IMDA Open Innovation Platform | Call 28 | March 2026

Version	1.0 — Initial Release
Status	READY FOR DEVELOPMENT
Date	March 2026
Stack	React (Frontend) + FastAPI (Backend)
Hosting	AWS (Singapore Region) — Dockerized
Submission Deadline	17 April 2026

Table of Contents

1. Project Overview

1.1 Background & Context

NUH's Medical Affairs (Education) department coordinates clinical training across four disciplines — Medicine, Allied Health, Nursing, and other Training Programmes — serving students from partner institutions including NUS, SIT, Singapore Polytechnic, and PSB Academy. The department manages student onboarding and postings management, tutor engagement, teaching activity tracking, feedback collection, and reporting.

The current operational model is heavily manual and fragmented across Excel spreadsheets, FormSG submissions, email threads, and disconnected systems. This creates significant operational burden and data quality issues that limit the department's ability to scale.

1.2 Problem Statement

> *"How might we streamline and standardise fragmented clinical education workflows through a single, unified system that improves operational efficiency and reduces manual effort across disciplines?"*

1.3 Solution Summary

ClinEdOps is a unified, web-based SaaS platform built on React (frontend) and FastAPI (backend), deployed on AWS Singapore via Docker. It replaces fragmented manual processes with a single source of truth for student and tutor lifecycle management, teaching hour tracking, structured feedback collection, automated reporting, and role-based access control — covering all four NUH clinical education disciplines.

1.4 Success Metrics

Metric	Baseline (Current)	Target
Manual data entry / tracking time	Benchmark via user survey	50% reduction
Data accuracy & completeness	Inconsistent across disciplines	≥ 90%
Report generation time	~5 minutes per report	< 1 minute (80% reduction)
Survey completion rate	Baseline via audit	≥ 30% increase
Staff / student satisfaction	Baseline via survey	Measurable improvement

2. User Personas

The platform serves five distinct user types, each with specific needs, workflows, and access levels.

2.1 Persona 1 — Programme Administrator (e.g., Sarah)

Role	Allied Health / Nursing / Training Programme Coordinator
Department	Medical Affairs (Education)
Institutions Served	SIT, NUS, Singapore Polytechnic, PSB Academy
Technical Proficiency	Intermediate — comfortable with Excel, FormSG, email workflows

Key Responsibilities

- Manages full student lifecycle: onboarding, posting assignment, offboarding
- Tracks teaching hours per tutor across multiple students
- Collects, downloads, and reformats feedback from FormSG into reports
- Prepares tutor evaluation reports and processes billing data
- Coordinates across departments, institutions, and disciplines

Pain Points

- Spends days during peak periods manually consolidating Excel, CSV, email, and PDF data
- No easy way to verify accuracy of late or inconsistently submitted teaching hours
- Must manually track feedback completion and send individual reminders
- Report generation for appraisals and awards is time-consuming and error-prone

Platform Needs

- Single dashboard: all postings, tutor assignments, teaching hours, feedback status
- Automated anomaly flags on teaching hour submissions
- One-click, customisable report generation for appraisals, billing, and awards
- Batch upload of student and tutor data from Excel and FormSG
- Automated reminder system for pending feedback submissions

2.2 Persona 2 — Medical Tutor / Clinician (e.g., Dr. Aisha)

Role	Supervising Doctor / Clinician Educator
Department	Various Clinical Departments (Medicine)
Student Interactions	Multiple students across different rotations and specialties

Technical Proficiency	Low to intermediate — prefers minimal-click interfaces

Key Responsibilities

- Supervises medical students across different rotations
- Conducts scheduled formal teaching sessions and ad hoc consultations
- Logs teaching hours and reviews consolidated student feedback

Pain Points

- Cumbersome, multi-screen teaching hour logging system
- Ad hoc hours must sometimes be submitted separately via email
- Students optionally input hours, causing discrepancies
- No real-time visibility into whether hours have been approved
- No consolidated view of teaching contributions or feedback

Platform Needs

- Fast, simple session logging for both scheduled and ad hoc teaching
- Real-time personal dashboard: accumulated hours + approval status
- Automatically compiled feedback summaries with strength/improvement view
- Mobile-responsive interface for quick logging between clinical duties

2.3 Persona 3 — Department Head / Senior Oversight (e.g., Bryan)

Role	Head of Medical Affairs (Education)
Scope	All four disciplines: Medicine, Allied Health, Nursing, Training Programmes
Decision Type	Strategic — programme quality, governance, resource allocation
Technical Proficiency	Low — needs clean dashboards, not raw data

Key Responsibilities

- Oversees clinical education quality across all disciplines
- Reviews job shadowing applications, shortlists, and assigns mentors
- Tracks positive feedback for tutor recognition and award nominations
- Identifies improvement opportunities from negative feedback trends
- Responsible for compliance, governance, and accreditation reporting

Pain Points

- Mentor-student matching tracked in spreadsheets with no real-time status
- Positive/negative feedback must be manually extracted and reformatted

- Data is fragmented across disciplines — no unified cross-discipline view
- Strategic decisions rely on manually compiled, potentially outdated reports

Platform Needs

- Cross-disciplinary real-time dashboards: teaching hours, feedback trends, compliance rates
- Drill-down capability from aggregate to individual tutor/student level
- Automated quality flags — surface low-scoring feedback for review
- Award nomination pipeline — one-click export of positive feedback summaries

2.4 Persona 4 — Student

Role	Clinical Posting Student
Disciplines	Medicine, Allied Health (Physiotherapy, OT, Speech Therapy), Nursing
Institutions	NUS, SIT, Singapore Polytechnic, PSB Academy
Technical Proficiency	High — comfortable with web and mobile apps

Pain Points

- Survey fatigue — currently receives a feedback request for every single tutor interaction in a day
- No centralised view of own posting schedule, tutor assignments, and feedback status
- Teaching hours submitted optionally, leading to inaccurate records

Platform Needs

- Consolidated, batched feedback surveys (not per-interaction spam)
- Personal dashboard: posting schedule, assigned tutors, feedback completion status
- Simple interface to view and confirm teaching hour logs

2.5 Persona 5 — Super Administrator (IT / Platform Owner)

Role	NUH IT / Platform Owner
Responsibility	System configuration, user management, audit compliance, integrations
Technical Proficiency	High — manages RBAC, integrations, system settings

Platform Needs

- Full RBAC management: create/edit/delete roles and permissions

- Full audit trail with timestamps for all system actions
- Integration management: NUHS Jobplan Portal, NUS BLUE, TAS
- System health monitoring, access logs, data export for compliance

3. Scope — What Needs to Be Built

3.1 In-Scope Modules

Module 1: Student & Tutor Lifecycle Management

Centralised record system for all students and tutors across all four disciplines.

Features

- Student profiles: name, institution, discipline, cohort, academic cycle, current posting status
- Tutor profiles: name, department, specialty, billable rate, active/inactive status
- Assign students to departments, postings, tutors, and academic cycles
- Status tracking: Pending Onboarding → Active Posting → Completed / Offboarded
- Batch upload from Excel (.xlsx) and FormSG exports (CSV/JSON)
- Duplicate detection, error highlighting, and anomaly flagging on batch imports
- Offline offboarding workflows with final data capture
- Historical record retention for accreditation and audit

Module 2: Teaching Hours Tracking

Standardised mechanism for tutors to log, submit, and get approval for teaching sessions.

Features

- Session logging: date, time, duration, type (scheduled/ad hoc), linked student(s), department
- Bulk session creation for recurring scheduled sessions
- Student-side optional hour confirmation / view
- Multi-level approval workflow: Tutor submits → Supervisor reviews → Admin approves
- Automated anomaly detection: flag duplicate sessions, unusually high hours, sessions outside posting period
- Map approved hours to tutor billable rates for compensation processing
- Real-time dashboard of approved, pending, and flagged hours by tutor/department/discipline
- Export teaching hours summary for billing and payroll

Module 3: Configurable Survey & Feedback Engine

Flexible survey system that eliminates survey fatigue while ensuring quality data collection.

Features

- Survey template builder: configurable by discipline, posting type, semester, or academic cycle
- Scoring matrix: 5-point Likert scale as standard

- Compulsory text fields auto-triggered when score falls below configurable threshold (e.g., < 3/5)
- Low-score alerts: auto-flag to programme admin and supervisor for follow-up
- Batched / consolidated surveys: group multiple tutor interactions into a single daily or weekly survey (eliminates per-interaction fatigue)
- Configurable survey cadence: per-posting, midpoint check-in, end-of-posting
- Midpoint feedback form: structured qualitative check-in at posting midpoint
- Automated reminder system: configurable reminder intervals for incomplete submissions
- Real-time submission rate monitoring: by cohort, posting, discipline
- Positive/negative feedback categorisation and filtering tools

Module 4: Reporting & Analytics

Comprehensive reporting engine with real-time dashboards and one-click export.

Features

- Real-time dashboards (role-specific views): teaching hours by tutor/dept/discipline, survey completion rates, feedback quality indicators, posting status overview
- Drill-down: click any aggregate metric to see individual-level detail
- Pre-built report templates: Faculty Appraisal, Tutor Billing, Teaching Award Nominations, Programme Quality Review, Accreditation Report
- Custom report builder: select fields, filters, date ranges, disciplines, departments
- Report generation target: < 1 minute end-to-end
- Export formats: PDF, Excel (.xlsx), CSV
- Scheduled report delivery: auto-generate and email reports on configurable schedule
- Comparative analytics: semester-on-semester, year-on-year teaching quality trends

Module 5: Notifications & Alerts

Features

- In-app notification centre with read/unread status
- Email notifications (configurable): pending feedback, approval required, low scores flagged, upcoming deadlines
- Push reminders for incomplete survey submissions
- Escalation alerts: if feedback overdue past configurable threshold, escalate to supervisor
- Admin broadcast: send announcements to student or tutor groups

Module 6: Role-Based Access Control (RBAC)

Roles

Role	Description	Key Permissions
Super Admin	NUH IT / Platform Owner	Full system access, user management, RBAC config, audit logs, integrations

Programme Admin	Department-level coordinator (e.g., Sarah)	Manage students/tutors in assigned discipline, generate reports, view all feedback
Supervisor	Senior clinician overseeing tutors	Approve teaching hours, view tutor feedback summaries, flag concerns
Tutor / Clinician	Teaching clinician (e.g., Dr. Aisha)	Log teaching hours, view own feedback, see assigned students
Student	Clinical posting student	View own schedule, complete surveys, confirm teaching hours

Additional RBAC Features

- Customisable permission sets per role (super admin can modify defaults)
- Approval hierarchy configuration: define who approves what within each workflow
- Discipline-scoped access: programme admins see only their assigned discipline by default
- Cross-discipline view for department heads and above
- Time-limited guest access for visiting educators or external accreditors

Module 7: Audit Trail

Features

- Immutable audit log of all create, update, delete, and approve actions
- Captures: actor (user ID + role), action type, timestamp, affected record, before/after values
- Filterable and searchable audit log for admins
- Exportable for compliance, PDPA audit, or accreditation review
- Retention policy: configurable (default 7 years)

Module 8: Batch Import / Data Management

Features

- Excel (.xlsx) and CSV import for student and tutor data at start of each academic cycle
- Field mapping interface: map import columns to system fields
- Pre-import validation: check for duplicates, missing required fields, format errors
- Import preview with row-level error highlighting before committing
- Import history log: who imported what, when, and how many records
- FormSG webhook / export ingestion for real-time form submissions

Module 9: System Integrations

Target Integrations

System	Integration Type	Data Flow	Priority

NUHS Jobplan Portal	REST API / DB sync	Tutor schedules, job plan hours into ClinEdOps	High
NUS BLUE System	REST API	Student records, academic calendar, posting schedules	High
Teaching Activities System (TAS)	REST API / CSV export	Teaching activity data bi-directional sync	High
FormSG	Webhook / CSV	Student/tutor data import, form submission ingestion	Medium
AWS SES	SDK	Outbound email notifications and report delivery	High
AWS S3	SDK	File storage: exports, attachments, audit logs	High

3.2 Out of Scope (MVP / POC)

- Student grading or summative academic assessment management
- Financial payment processing (billing data export is in scope; payment gateway is not)
- Patient records, clinical EHR, or EMR integration
- Native iOS / Android mobile app (mobile-responsive web is in scope)
- Full multi-tenancy rollout to other NUHS-affiliated institutions (post-POC roadmap item)
- Live video or teleconferencing for virtual teaching sessions

4. Technical Architecture

4.1 Architecture Overview

ClinEdOps follows a modern three-tier architecture: a React single-page application (SPA) frontend, a FastAPI Python backend exposing a RESTful API, and a PostgreSQL relational database. All services are containerised via Docker and orchestrated with Docker Compose for local development, deploying to AWS via ECS Fargate for production.

4.2 Technology Stack

Layer	Technology	Rationale
Frontend	React 18 + TypeScript	Component-based SPA; strong ecosystem; TypeScript for type safety
UI Library	Ant Design (antd) or Shadcn/UI + Tailwind CSS	Professional, accessible components; fast UI development
State Management	React Query (TanStack Query) + Zustand	Server-state caching + lightweight client-state management
Routing	React Router v6	Client-side routing with protected routes and role guards
Charts & Dashboards	Recharts or Apache ECharts	Rich, configurable charts for teaching hours and analytics
Backend Framework	FastAPI (Python 3.12)	Async, high-performance; auto OpenAPI docs; type hints
ORM	SQLAlchemy 2.0 + Alembic	Robust ORM with migrations; async support via asyncpg
Database	PostgreSQL 16	Relational integrity for complex role/student/tutor relationships
Authentication	FastAPI-Users + JWT (RS256)	Secure stateless auth; refresh token rotation
Task Queue	Celery + Redis	Background jobs: report generation, email dispatch, reminders
File Storage	AWS S3 (Singapore region)	Scalable object storage for exports, attachments
Email	AWS SES	Transactional email: notifications, reports, reminders
Containerisation	Docker + Docker Compose	Reproducible builds; environment parity
Orchestration (Prod)	AWS ECS Fargate	Serverless containers; auto-scaling; no server management
Load Balancer	AWS ALB	HTTPS termination; path-based routing to services
CI/CD	GitHub Actions	Automated test, lint, build, and deploy pipeline

Reverse Proxy	Nginx (in container)	Static asset serving; API proxy; compression
Monitoring	AWS CloudWatch + Sentry	Infrastructure metrics + frontend/backend error tracking
Secret Management	AWS Secrets Manager	Secure credential storage for DB, API keys, JWT secrets

4.3 System Architecture Diagram (Description)

The production architecture on AWS Singapore (ap-southeast-1) is structured as follows:

- Route 53 DNS → CloudFront CDN → React SPA (served from S3 + CloudFront)
- API requests: CloudFront → AWS ALB → ECS Fargate (FastAPI containers) in private subnet
- FastAPI → RDS PostgreSQL (Multi-AZ, private subnet)
- FastAPI → ElastiCache Redis (Celery broker + caching)
- Celery workers (separate ECS task) → AWS SES (emails) + S3 (file storage)
- All ECS tasks in VPC private subnets; ALB in public subnet
- AWS WAF on ALB for rate limiting and injection protection
- CloudWatch for logs, metrics, alarms; Sentry SDK in both frontend and backend

4.4 Docker Setup

The following Docker Compose services are defined for local development:

```
services:
  frontend:  # React app — port 3000
  backend:   # FastAPI app — port 8000
  db:        # PostgreSQL 16 — port 5432
  redis:     # Redis 7 — port 6379 (Celery broker + cache)
  celery:    # Celery worker (same image as backend)
  nginx:     # Reverse proxy — port 80/443
```

Each service has a dedicated Dockerfile. Multi-stage builds are used for production images to minimise image size. Environment variables are managed via .env files locally and AWS Secrets Manager in production.

4.5 Repository Structure

```
clinedops/
├── frontend/               # React TypeScript app
│   ├── src/
│   │   ├── components/     # Reusable UI components
│   │   ├── pages/          # Route-level page components
│   │   ├── hooks/          # Custom React hooks
│   │   ├── store/          # Zustand state stores
│   │   ├── api/            # React Query API hooks (Axios)
│   │   ├── types/          # TypeScript interfaces / types
```

```
│   │   │   └── utils/          # Helpers, formatters, validators
│   │   ├── Dockerfile
│   │   └── nginx.conf          # Nginx config for SPA routing
├── backend/                    # FastAPI Python app
│   ├── app/
│   │   ├── api/                # Route handlers (versioned: /api/v1/)
│   │   │   ├── auth/
│   │   │   ├── students/
│   │   │   ├── tutors/
│   │   │   ├── postings/
│   │   │   ├── teaching_hours/
│   │   │   ├── surveys/
│   │   │   ├── feedback/
│   │   │   ├── reports/
│   │   │   ├── notifications/
│   │   │   └── admin/
│   │   ├── models/             # SQLAlchemy ORM models
│   │   ├── schemas/            # Pydantic request/response schemas
│   │   ├── services/           # Business logic layer
│   │   ├── tasks/              # Celery async tasks
│   │   ├── integrations/       # External system connectors (BLUE, TAS, Jobplan)
│   │   ├── core/               # Config, security, dependencies
│   │   └── db/                 # DB session, base model, migrations
│   ├── migrations/             # Alembic migrations
│   ├── tests/                  # Pytest test suite
│   └── Dockerfile
├── docker-compose.yml          # Local dev orchestration
├── docker-compose.prod.yml     # Production overrides
├── .github/workflows/          # GitHub Actions CI/CD
└── infra/                      # AWS CDK / Terraform IaC
```

5. Database Schema

5.1 Core Tables

The following tables form the core data model. All tables include: id (UUID PK), created_at, updated_at, created_by (FK → users).

users

Column	Type	Constraints	Description
id	UUID	PK, default gen_random_uuid()	Primary key
email	VARCHAR(255)	UNIQUE, NOT NULL	Login email address
hashed_password	VARCHAR(255)	NOT NULL	Bcrypt hashed password
full_name	VARCHAR(255)	NOT NULL	Display name
role	ENUM	NOT NULL	super_admin / programme_admin / supervisor / tutor / student
discipline	ENUM	NULLABLE	medicine / allied_health / nursing / training
institution	VARCHAR(255)	NULLABLE	For students: NUS, SIT, SP, PSB Academy
is_active	BOOLEAN	DEFAULT TRUE	Soft deactivation
last_login_at	TIMESTAMPTZ	NULLABLE	Last successful login

students

Column	Type	Constraints	Description
id	UUID	PK	Primary key
user_id	UUID	FK → users, NULLABLE	Linked user account (if portal access granted)
student_code	VARCHAR(100)	UNIQUE, NOT NULL	Institution student ID
full_name	VARCHAR(255)	NOT NULL	Student full name
discipline	ENUM	NOT NULL	medicine / allied_health / nursing / training
institution	VARCHAR(255)	NOT NULL	Sending institution
programme	VARCHAR(255)	NOT NULL	e.g., Physiotherapy, MBBS, Nursing Diploma
academic_cycle_id	UUID	FK → academic_cycles	Current academic cycle

status	ENUM	NOT NULL	pending_onboard / active / completed / offboarded

tutors

Column	Type	Constraints	Description
id	UUID	PK	Primary key
user_id	UUID	FK → users	Linked user account
staff_code	VARCHAR(100)	UNIQUE, NOT NULL	NUH staff ID
full_name	VARCHAR(255)	NOT NULL	Tutor full name
department	VARCHAR(255)	NOT NULL	Clinical department
specialty	VARCHAR(255)	NULLABLE	Clinical specialty
discipline	ENUM	NOT NULL	Disciplines this tutor teaches
billable_rate	NUMERIC(10,2)	NULLABLE	Hourly billing rate (SGD)
is_active	BOOLEAN	DEFAULT TRUE	Active status

postings

Column	Type	Constraints	Description
id	UUID	PK	Primary key
student_id	UUID	FK → students, NOT NULL	Student on posting
department_id	UUID	FK → departments, NOT NULL	Host department
academic_cycle_id	UUID	FK → academic_cycles	Academic cycle
start_date	DATE	NOT NULL	Posting start date
end_date	DATE	NOT NULL	Posting end date
status	ENUM	NOT NULL	scheduled / active / completed / cancelled
primary_tutor_id	UUID	FK → tutors, NULLABLE	Primary assigned tutor

posting_tutors (junction)

Column	Type	Constraints	Description
posting_id	UUID	FK → postings	Posting reference
tutor_id	UUID	FK → tutors	Tutor reference

role_in_posting	VARCHAR(100)	NULLABLE	e.g., primary supervisor, ad hoc

teaching_sessions

Column	Type	Constraints	Description
id	UUID	PK	Primary key
tutor_id	UUID	FK → tutors, NOT NULL	Teaching tutor
posting_id	UUID	FK → postings, NULLABLE	Associated posting
department_id	UUID	FK → departments	Department where taught
session_date	DATE	NOT NULL	Date of session
start_time	TIME	NOT NULL	Session start time
duration_minutes	INTEGER	NOT NULL	Duration in minutes
session_type	ENUM	NOT NULL	scheduled / ad_hoc / consultation
description	TEXT	NULLABLE	Session description
status	ENUM	NOT NULL	draft / submitted / approved / flagged / rejected
approved_by	UUID	FK → users, NULLABLE	Approver user ID
approved_at	TIMESTAMPTZ	NULLABLE	Approval timestamp
billable_hours	NUMERIC(5,2)	NULLABLE	Computed from duration_minutes

session_students (junction)

Column	Type	Constraints	Description
session_id	UUID	FK → teaching_sessions	Session reference
student_id	UUID	FK → students	Student present in session
student_confirmed	BOOLEAN	DEFAULT FALSE	Student confirmed attendance

survey_templates

Column	Type	Constraints	Description
id	UUID	PK	Primary key
name	VARCHAR(255)	NOT NULL	Template name
discipline	ENUM	NULLABLE	Null = applies to all disciplines

posting_type	VARCHAR(255)	NULLABLE	e.g., clinical, shadow
survey_type	ENUM	NOT NULL	midpoint / end_of_posting / ad_hoc
questions	JSONB	NOT NULL	Array of question objects
low_score_threshold	INTEGER	DEFAULT 3	Score at or below which text comment required
is_active	BOOLEAN	DEFAULT TRUE	Is template currently in use

The questions JSONB field stores an array of objects: { id, text, type (rating/text/multi_choice), required, low_score_trigger }.

survey_submissions

Column	Type	Constraints	Description
id	UUID	PK	Primary key
template_id	UUID	FK → survey_templates	Template used
posting_id	UUID	FK → postings	Associated posting
student_id	UUID	FK → students	Submitting student
tutor_id	UUID	FK → tutors, NULLABLE	Tutor being evaluated (if applicable)
submitted_at	TIMESTAMPTZ	NULLABLE	Null if not yet submitted
responses	JSONB	NULLABLE	Array of { question_id, score, text_response }
overall_score	NUMERIC(3,1)	NULLABLE	Computed average score
has_low_scores	BOOLEAN	DEFAULT FALSE	Flag for low score alerts
status	ENUM	NOT NULL	pending / submitted / flagged

audit_logs

Column	Type	Constraints	Description
id	UUID	PK	Primary key
actor_id	UUID	FK → users, NOT NULL	User who performed action
actor_role	VARCHAR(100)	NOT NULL	Role snapshot at time of action
action	VARCHAR(100)	NOT NULL	e.g., CREATE, UPDATE, DELETE, APPROVE
resource_type	VARCHAR(100)	NOT NULL	e.g., student, teaching_session, survey
resource_id	UUID	NOT NULL	ID of affected record

before_state	JSONB	NULLABLE	Record state before change
after_state	JSONB	NULLABLE	Record state after change
ip_address	INET	NULLABLE	Client IP address
timestamp	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	Action timestamp

Additional Supporting Tables

- academic_cycles — id, name (e.g., AY2025/26 Sem 1), start_date, end_date, is_active
- departments — id, name, division, discipline, head_id (FK → users)
- notifications — id, user_id, type, title, body, is_read, created_at, action_url
- import_batches — id, uploaded_by, import_type, file_name, record_count, error_count, status, created_at
- report_definitions — id, name, type, config_json (fields, filters, sort), created_by, is_shared

6. API Specification

6.1 API Design Principles

- Base URL: /api/v1/
- All responses: JSON with standard envelope: { data, meta, errors }
- Authentication: Bearer JWT token in Authorization header
- Pagination: cursor-based for large datasets; limit/offset for simple lists
- Versioning: URL path versioning (/api/v1/, /api/v2/)
- Error format: { code, message, field (if validation), trace_id }
- Rate limiting: 1000 req/min per authenticated user; 100 req/min for public endpoints

6.2 Authentication Endpoints

Method	Path	Description	Auth Required
POST	/api/v1/auth/login	Email + password login; returns access + refresh tokens	No
POST	/api/v1/auth/refresh	Refresh access token using refresh token	Refresh token
POST	/api/v1/auth/logout	Invalidate refresh token	Yes
POST	/api/v1/auth/forgot-password	Send password reset email	No
POST	/api/v1/auth/reset-password	Reset password via token	No
GET	/api/v1/auth/me	Get current authenticated user profile	Yes
PATCH	/api/v1/auth/me	Update own profile details	Yes

6.3 Student Endpoints

Method	Path	Description	Roles
GET	/api/v1/students	List students (filtered, paginated)	Admin, Supervisor
POST	/api/v1/students	Create single student record	Admin
POST	/api/v1/students/batch	Batch create from Excel/CSV upload	Admin
GET	/api/v1/students/{id}	Get student detail with postings history	Admin, Supervisor, Student (own)
PATCH	/api/v1/students/{id}	Update student record	Admin
DELETE	/api/v1/students/{id}	Soft delete (deactivate) student	Super Admin

GET	/api/v1/students/{id}/postings	Get all postings for a student	Admin, Supervisor, Student (own)
GET	/api/v1/students/{id}/feedback	Get all feedback submissions for a student	Admin

6.4 Tutor Endpoints

Method	Path	Description	Roles
GET	/api/v1/tutors	List tutors (filtered, paginated)	Admin, Supervisor
POST	/api/v1/tutors	Create single tutor record	Admin
POST	/api/v1/tutors/batch	Batch create from Excel/CSV upload	Admin
GET	/api/v1/tutors/{id}	Get tutor detail with teaching summary	Admin, Supervisor, Tutor (own)
PATCH	/api/v1/tutors/{id}	Update tutor record	Admin
GET	/api/v1/tutors/{id}/teaching-hours	Get teaching hours summary for tutor	Admin, Supervisor, Tutor (own)
GET	/api/v1/tutors/{id}/feedback-summary	Get aggregated feedback for tutor	Admin, Supervisor, Tutor (own)
GET	/api/v1/tutors/{id}/billing	Get billing-ready hours export	Admin

6.5 Teaching Sessions Endpoints

Method	Path	Description	Roles
GET	/api/v1/teaching-sessions	List sessions (filtered by tutor/dept/status)	Admin, Supervisor, Tutor (own)
POST	/api/v1/teaching-sessions	Log a new teaching session	Tutor, Admin
GET	/api/v1/teaching-sessions/{id}	Get session detail	Admin, Supervisor, Tutor (own)
PATCH	/api/v1/teaching-sessions/{id}	Edit draft session	Tutor (own, draft only), Admin
POST	/api/v1/teaching-sessions/{id}/submit	Submit session for approval	Tutor
POST	/api/v1/teaching-sessions/{id}/approve	Approve submitted session	Supervisor, Admin
POST	/api/v1/teaching-sessions/{id}/reject	Reject session with reason	Supervisor, Admin
POST	/api/v1/teaching-sessions/{id}/flag	Flag session as anomalous	Admin

GET	/api/v1/teaching-sessions/summary	Aggregated hours by tutor/dept/discipline	Admin, Supervisor

6.6 Survey & Feedback Endpoints

Method	Path	Description	Roles
GET	/api/v1/survey-templates	List survey templates	Admin
POST	/api/v1/survey-templates	Create new survey template	Admin
PATCH	/api/v1/survey-templates/{id}	Edit survey template	Admin
GET	/api/v1/surveys	List survey submissions (filterable)	Admin, Supervisor
GET	/api/v1/surveys/pending	Get pending surveys for current user	Student
POST	/api/v1/surveys/{id}/submit	Submit survey response	Student
GET	/api/v1/surveys/{id}	Get survey submission detail	Admin, Supervisor, Student (own)
GET	/api/v1/surveys/completion-rates	Get completion rate stats by posting/cohort	Admin, Supervisor
GET	/api/v1/feedback/summary	Aggregated feedback scores and comments	Admin, Supervisor
GET	/api/v1/feedback/low-scores	List flagged low-score submissions	Admin, Supervisor

6.7 Reporting Endpoints

Method	Path	Description	Roles
GET	/api/v1/reports	List saved report definitions	Admin
POST	/api/v1/reports	Create new report definition	Admin
POST	/api/v1/reports/{id}/generate	Trigger async report generation	Admin
GET	/api/v1/reports/{id}/download	Download generated report file	Admin
GET	/api/v1/reports/appraisal	Generate faculty appraisal report (async)	Admin
GET	/api/v1/reports/billing	Generate tutor billing report (async)	Admin
GET	/api/v1/reports/awards	Generate teaching awards report (async)	Admin
GET	/api/v1/reports/accreditation	Generate accreditation summary (async)	Admin
GET	/api/v1/dashboard/overview	Real-time overview dashboard data	Admin, Supervisor, HOD

GET	/api/v1/dashboard/teaching-hours	Teaching hours dashboard data	Admin, Supervisor, Tutor
GET	/api/v1/dashboard/feedback	Feedback analytics dashboard data	Admin, Supervisor

7. Frontend Screens & UI Specification

7.1 Application Layout

- Persistent left sidebar navigation (collapsible) with role-specific menu items
- Top header bar: breadcrumbs, notifications bell (with badge count), user avatar + dropdown
- Main content area: full-width, responsive
- Role-based navigation: menu items shown/hidden based on authenticated user role
- Mobile-responsive: sidebar collapses to bottom tab bar on mobile

7.2 Screen List by Module

Authentication

- Login — email + password; forgot password link; NUH branding
- Forgot Password — email entry; success confirmation
- Reset Password — token from email; new password entry + confirm
- First Login / Onboarding — forced password change + profile completion

Dashboard (Role-Specific)

- Admin Dashboard — KPI cards (active students, pending hours, survey completion %, flagged items); quick action buttons; recent activity feed
- HOD Dashboard — cross-discipline summary; teaching quality heatmap; compliance status; trend charts
- Tutor Dashboard — my pending hours, my approved hours this cycle, my feedback summary, upcoming assigned students
- Student Dashboard — my active posting, pending surveys, assigned tutors, posting timeline

Student Management

- Students List — filterable (discipline, institution, status, academic cycle), sortable table, bulk actions, export button
- Student Detail — profile card; posting history timeline; feedback received; teaching hours; audit trail
- Add Student (single) — form with validation
- Batch Import Students — file upload; field mapping interface; validation preview table; import confirmation

Tutor Management

- Tutors List — filterable, sortable; shows current teaching load; status badges

- Tutor Detail — profile; teaching hours by cycle; feedback summary; billing summary; assigned students
- Add Tutor (single) — form
- Batch Import Tutors — file upload with validation

Postings Management

- Postings List — calendar view + table view; filter by department, discipline, date range, status
- Posting Detail — student info; assigned tutors; teaching sessions; survey status; timeline
- Create / Edit Posting — date picker; student search; tutor assignment; department selector

Teaching Hours

- Log Session (Tutor) — date/time pickers; session type selector; student multi-select; description; submit/save draft
- My Sessions (Tutor) — personal list of sessions; status tags (draft/submitted/approved/rejected); hours summary
- Sessions Admin View — all sessions across tutors; filter by status, dept, tutor; bulk approve; anomaly flag badges
- Session Detail — full session info; approval history; student list; audit trail
- Teaching Hours Dashboard — bar charts by tutor/department; filter by date/discipline; export

Surveys & Feedback

- Survey Templates List — list of active templates; clone; edit; deactivate
- Survey Template Editor — drag-and-drop question builder; question types (rating, text, multi-choice); low score threshold setting
- My Pending Surveys (Student) — list of surveys with due dates; progress indicator; one-click to start
- Survey Form (Student) — clean form with progress bar; Likert scale for ratings; conditional text fields on low scores; submit confirmation
- Feedback Admin View — completion rate progress bars by cohort; flagged submissions list; feedback comment browser with sentiment filter
- Feedback Summary (Tutor) — personal view of aggregated scores; comment excerpts; trend over time

Reports

- Reports Centre — list of available report templates; generate + download buttons; recent exports history
- Custom Report Builder — field picker; filter panel; date range; preview table; generate + download
- Scheduled Reports — list of scheduled report deliveries; enable/disable; configure recipients

Notifications

- Notification Centre — full list with filter (unread, type, date); mark all read; click-through to relevant record

Admin Panel (Super Admin only)

- User Management — list all users; create/edit/deactivate; role assignment
- RBAC Configuration — role permission matrix editor
- Academic Cycle Management — create/manage academic cycles; activate/close cycles
- Department Management — add/edit departments; assign department heads
- Audit Log Viewer — full system audit log; searchable; filterable; exportable
- Integration Status — status of BLUE, TAS, Jobplan Portal connections; sync logs; manual re-sync
- System Settings — email config, notification defaults, PDPA data retention policy

8. Non-Functional Requirements

Category	Requirement	Specification
Performance	API response time	P95 < 500ms for standard queries; < 2s for report generation triggers
Performance	Report generation	< 60 seconds end-to-end for all pre-built report types
Performance	Concurrent users	Support 500+ concurrent users without degradation
Reliability	Uptime SLA	99.5% uptime during business hours (0800–2200 SGT)
Reliability	Backup	Daily automated PostgreSQL backups; 30-day retention; point-in-time recovery
Scalability	Horizontal scaling	ECS Fargate auto-scaling: scale out at 70% CPU; scale in at 30% CPU
Scalability	Database	RDS PostgreSQL with read replica for reporting queries
Security	Authentication	JWT RS256; access token 15 min TTL; refresh token 7 day TTL with rotation
Security	Transport	HTTPS enforced on all endpoints; TLS 1.2+ minimum
Security	PDPA compliance	Data stored in Singapore; access logs; data export on request; retention policy
Security	Input validation	All inputs validated server-side; parameterised SQL queries (no raw SQL)
Security	WAF	AWS WAF rules: rate limiting, SQL injection, XSS protection on ALB
Hosting	Cloud region	AWS ap-southeast-1 (Singapore) — mandatory for data residency
Accessibility	WCAG	WCAG 2.1 Level AA compliance for key user-facing screens
Browser Support	Minimum	Chrome 110+, Firefox 110+, Safari 16+, Edge 110+
Mobile	Responsive	Full functionality on viewport width >= 375px (iPhone SE breakpoint)
Internationalisation	Language	English only for MVP; architecture must support i18n
Data Integrity	Transactions	All multi-step write operations wrapped in database transactions

9. Security Design

9.1 Authentication & Authorisation

- JWT RS256 asymmetric signing — private key in AWS Secrets Manager; public key served at /api/v1/auth/jwks
- Access tokens: 15-minute TTL; stored in memory (not localStorage)
- Refresh tokens: 7-day TTL; HTTP-only secure cookie; server-side rotation on use
- Every API endpoint decorated with role permission check via FastAPI dependency injection
- RBAC enforced at service layer: data queries filtered by actor's role and discipline scope

9.2 Data Security

- All database connections via SSL/TLS; RDS in private subnet (no public endpoint)
- Passwords hashed with bcrypt (cost factor 12)
- Sensitive config (DB passwords, JWT keys, SES credentials) in AWS Secrets Manager — never in environment variables or code
- S3 bucket: private, server-side encryption (SSE-S3), no public access, pre-signed URLs for downloads (15-min expiry)
- All API inputs validated with Pydantic; SQL injection prevention via SQLAlchemy ORM

9.3 PDPA Compliance

- Data stored exclusively in AWS Singapore (ap-southeast-1)
- Access logs retained for 7 years (configurable)
- Student/tutor data export available on request (admin function)
- Data deletion workflow for departed staff/students (soft delete + configurable hard delete schedule)
- Privacy notice presented at first login

10. Implementation Roadmap

10.1 Phased Delivery Plan

Given the challenge deadline of 17 April 2026, the recommended approach is a 4-week sprint to build a functional MVP/prototype. Full production deployment follows post-challenge.

Phase	Duration	Scope	Deliverable
0 — Foundations	Week 1	Repo setup, Docker Compose, DB schema, auth system, RBAC skeleton, CI/CD pipeline	Running app skeleton with login
1 — Core Data	Week 1-2	Student/tutor CRUD, batch import, posting management, academic cycle management, admin panel	Working data management layer
2 — Teaching Hours	Week 2	Session logging, approval workflow, anomaly detection, tutor dashboard, hours summary	Full teaching hours module
3 — Surveys & Feedback	Week 3	Template builder, survey engine, batched survey dispatch, reminder system, feedback analytics	Feedback collection working
4 — Reporting & Dashboards	Week 3-4	Dashboard KPIs, report generation, export (PDF/Excel/CSV), scheduled reports	Report centre complete
5 — Polish & Demo	Week 4	UI polish, seed demo data, end-to-end testing, performance review, demo script preparation	Challenge submission ready
Post-POC — Production	TBD	AWS ECS Fargate deployment, integrations (BLUE/TAS/Jobplan), security hardening, multi-institution rollout	Production deployment

10.2 MVP Scope (For Challenge Submission)

The MVP must demonstrate all core workflows end-to-end with realistic demo data. The following are mandatory for the prototype:

- Authenticated login with at least 3 roles (Admin, Tutor, Student)
- Student and tutor record management with batch import

- Posting creation and assignment
- Teaching session logging and approval workflow
- Configurable survey with batched dispatch and submission
- At least one real-time dashboard with live data
- At least one report generation (appraisal or billing) with export
- Notification system (in-app minimum)
- Audit trail visible to admin

10.3 Post-Challenge Roadmap

- Phase A: AWS production deployment (ECS Fargate, RDS, ElastiCache, ALB, CloudFront)
- Phase B: System integrations — NUHS Jobplan Portal, NUS BLUE, TAS
- Phase C: Advanced analytics — trend analysis, benchmarking, sentiment analysis on open-text feedback
- Phase D: Mobile-first PWA optimisation; optional native mobile app
- Phase E: Multi-institution expansion to other NUHS-affiliated hospitals

11. Testing Strategy

11.1 Backend Testing (pytest)

- Unit tests: service layer functions, Pydantic schema validation, utility functions
- Integration tests: API endpoint tests with test database (PostgreSQL in Docker)
- Auth tests: JWT generation/validation, role permission enforcement
- Async task tests: Celery task unit tests with mocked brokers
- Coverage target: ≥ 80% line coverage on services/ and api/ directories

11.2 Frontend Testing (Vitest + React Testing Library)

- Component unit tests: form components, table components, dashboard widgets
- Integration tests: form submission flows, role-based rendering
- E2E tests (Playwright or Cypress): full user journeys — login, log session, submit survey, generate report

11.3 CI/CD Testing Gates

- All PRs: lint (ESLint + Ruff), type check (tsc + mypy), unit tests must pass
- Main branch: full test suite including integration tests
- Deploy gate: zero test failures before deploy to staging or production

12. Environment Configuration

12.1 Backend Environment Variables

```
# Database
DATABASE_URL=postgresql+asyncpg://user:pass@db:5432/clinedops
# Auth / JWT
JWT_PRIVATE_KEY=<RS256 private key path or ARN>
JWT_PUBLIC_KEY=<RS256 public key path or ARN>
ACCESS_TOKEN_EXPIRE_MINUTES=15
REFRESH_TOKEN_EXPIRE_DAYS=7
# Redis / Celery
REDIS_URL=redis://redis:6379/0
# AWS
AWS_REGION=ap-southeast-1
AWS_S3_BUCKET=clinedops-files
AWS_SES_FROM_EMAIL=noreply@clinedops.nuh.sg
# App
APP_ENV=development  # development | staging | production
CORS_ORIGINS=["http://localhost:3000"]
SENTRY_DSN=<sentry DSN>
```

12.2 Frontend Environment Variables

```
VITE_API_BASE_URL=http://localhost:8000/api/v1
VITE_APP_ENV=development
VITE_SENTRY_DSN=<sentry DSN>
```

13. Open Questions & Assumptions

13.1 Assumptions

- NUH will provide test credentials and sample data (anonymised) for BLUE, TAS, and Jobplan Portal integrations during prototype development
- The prototype will use seed/demo data for the challenge submission demonstration
- Email delivery for notifications will use AWS SES sandbox mode during prototype phase
- Single-tenancy is sufficient for the POC; multi-tenancy architecture to be designed for post-challenge rollout
- English is the only required language for the prototype
- NUH staff will use desktop browsers primarily; students may use mobile

13.2 Open Questions

- What is the exact API/data format exposed by NUHS Jobplan Portal, NUS BLUE, and TAS for integration?
- Is SSO (e.g., NUH Azure AD / Singpass) required for the prototype, or is email/password sufficient?
- What is the exact billable rate structure — flat rate per tutor, tiered by session type, or negotiated per contract?
- Should students be able to self-register, or are they always admin-provisioned?
- What is the preferred report output format — Excel, PDF, or both required for all report types?
- Are there any existing NUH branding/UI guidelines (logo, colour palette, typography) to follow?
- Should the prototype support both Allied Health and Medical disciplines fully, or focus on one for the demo?

14. Challenge Evaluation Alignment

The following maps the solution design decisions directly to the IMDA evaluation criteria:

Evaluation Criterion	Weight	How ClinEdOps Addresses It
Solution Fit — Relevance	30%	Directly addresses every pain point in the brief: student lifecycle, teaching hours, feedback, reporting, survey fatigue, audit trail
Solution Readiness — Maturity	15%	React + FastAPI + PostgreSQL + Docker is a proven, production-grade stack; deployable to AWS in days
Solution Readiness — Scalability	15%	ECS Fargate auto-scaling; RDS read replicas; Redis caching; architecture supports multi-institution post-POC
Solution Advantage — Innovation	20%	Batched survey engine eliminates survey fatigue; anomaly detection on teaching hours; ML-ready feedback analytics pipeline; configurable without custom coding
Company Profile — Traction	10%	To be addressed in proposal section
Team Experience	10%	To be addressed in proposal section

Appendix A: Glossary

Term	Definition
Academic Cycle	A defined semester or academic year period (e.g., AY2025/26 Sem 1)
Allied Health	Non-physician clinical professions: physiotherapy, occupational therapy, speech therapy
BLUE System	NUS learning management system extended to NUH for medical education
ClinEdOps	Project codename: Clinical Education Operations Platform
FormSG	Singapore Government form builder used for data collection
Jobplan Portal	NUHS internal portal for managing clinician job plans and schedules
PDPA	Personal Data Protection Act — Singapore's primary data protection legislation
POC / Prototype	Proof of Concept — functional demonstration of the platform for the challenge
Posting	A defined period where a student is assigned to a clinical department
RBAC	Role-Based Access Control — permissions tied to user roles, not individuals
TAS	Teaching Activities System — NUH system for recording teaching activities
Survey Fatigue	Reduced response rates caused by too many/frequent survey requests

Appendix B: Key Contacts & Stakeholders

To be populated upon project kickoff with NUH Medical Affairs team contacts, IT department integration contacts, and participating institution liaisons.

Document prepared for NUH IMDA Challenge Call 28 — March 2026
ClinEdOps PRD v1.0 | CONFIDENTIAL